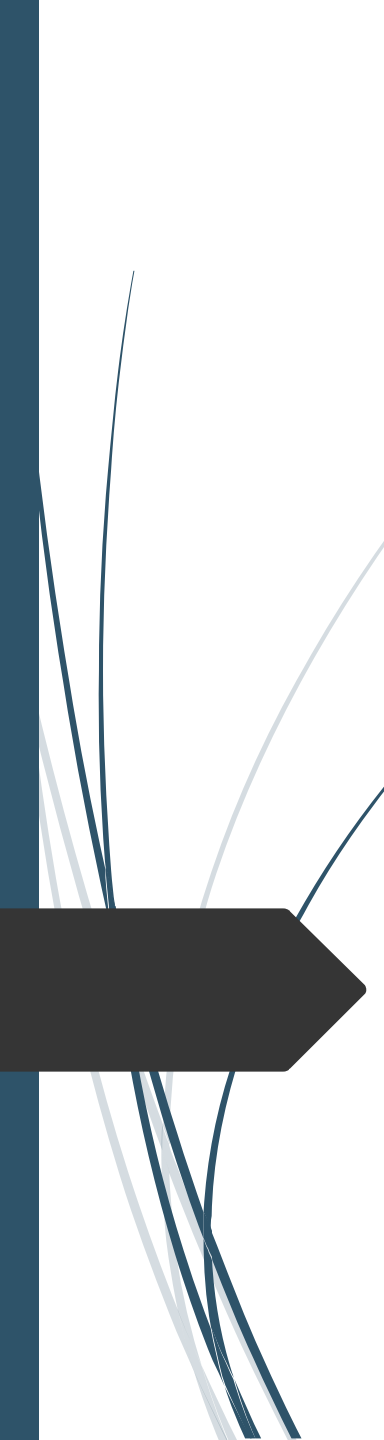




Basi di Dati - Modulo Laboratorio
A.a. 2025-2026

Lezione 2: Interrogazioni SQL

Dr. Beatrice Amico



Inserimento e aggiornamento di tuple

```
INSERT INTO Tabella [ ( Attributi ) ]  
VALUES( Valori )
```

oppure

```
INSERT INTO Tabella [ ( Attributi )]  
(SELECT ...)
```



Inserimento e aggiornamento di tuple

```
UPDATE NomeTabella  
SET Attributo = <  
Espressione |  
SELECT ... |  
NULL |  
DEFAULT >  
[ WHERE Condizione ]
```



Query SQL

```
SELECT ListaAttributi  
FROM ListaTabelle  
[ WHERE Condizione ]
```

```
SELECT [ DISTINCT ]  
[ * | expression [[ AS] output_name ] [,  
...] ]  
FROM from_item [, ...]  
[ WHERE condition ]  
[ GROUP BY grouping_element [, ...] ]  
[ HAVING condition [, ...] ]  
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT  
] other_select ]  
[ ORDER BY expression [ ASC | DESC | USING  
operator ]]
```

Calusola SELECT

```
SELECT [ DISTINCT ]  
[{ * | expression [[ AS] output_name ]} [, ...]]
```

- * è un'abbreviazione per indicare tutti gli attributi delle tabelle.
- **expression** è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- **output_name** è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione **expression** nella relazione risultato.
- **DISTINCT**: se presente richiede l'eliminazione delle tuple duplicate.



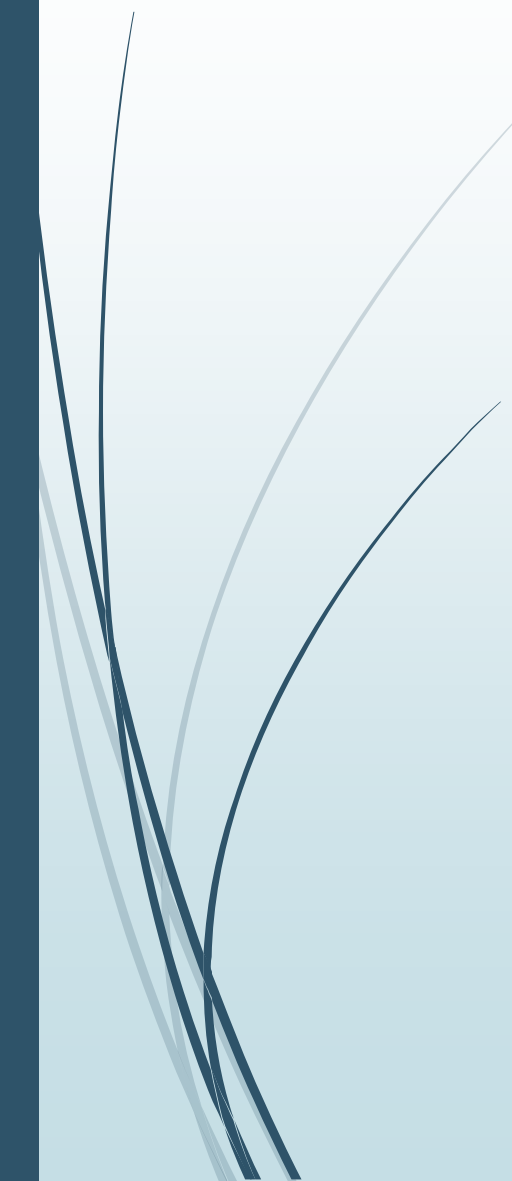
Clausola SELECT

L'esecuzione di una **SELECT** produce una relazione che:

- ha come schema tutti gli attributi elencati nella clausola **SELECT**;
- ha come contenuto tutte le tuple ottenute proiettando sugli attributi indicati nella **SELECT** provenienti dalle tabelle del **FROM** (prodotto cartesiano), limitate a quelle che soddisfano le eventuali condizioni specificate nelle clausole **WHERE**, **GROUP BY** e **HAVING**.



Target list



```
SELECT *  
FROM Tabella
```

Selezione di tutti gli attributi
delle tabelle indicate nella
clausola FROM

```
SELECT Attributo1, Attributo2  
FROM Tabella
```

Selezione di tutti gli attributi
delle tabelle indicate nella
clausola FROM

Clausola SELECT

Nome e reddito delle persone con meno di trenta anni.

$\text{PROJ}_{\text{Nome, Reddito}}(\text{SEL}_{\text{Eta} < 30}(\text{Persone}))$

```
SELECT Nome, Reddito  
FROM Persone  
WHERE Eta < 30
```

Nome	Reddito
Andrea	21
Aldo	15
Filipo	30

Persone

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filipo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

Clausola FROM

```
FROM from_item [, ...]
```

1. table_name [[AS] alias [(column_alias [, ...])]]
 - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nel SELECT questi attributi sono identificati antepoendo il nome della tabella e un '.':
nomeTabella.nomeAttributo
2. (other_select) [AS] nomeRisultato [(column_alias [...])]
 - è una **SELECT** innestata. Il risultato della **SELECT** interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
3. from_item [**NATURAL**] join_type from_item [**ON** condition]
 - è la clausola JOIN.
 - Metodo più sofisticato di 1) che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. **join_type** specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti.

Clausola FROM: uso di ALIAS

SQL consente di associare un nome alternativo (alias) alle tabelle che compaiono come argomento della clausola **FROM**.

```
table_name AS alias [(column_alias [, ...])]
```

```
SELECT Reddito, Telefono  
FROM Reparti, Persona  
WHERE ... ;
```

```
SELECT P.Reddito, R.Telefono  
FROM Reparti as R, Persona ad P  
WHERE ... ;
```

Utile quando è necessario specificare la provenienza dell'attributo, perchè nella base di dati sono presenti più attributi con lo stesso nome.

Target list: uso di ALIAS

```
SELECT P.Nome as NPersona, P.Reddito as RPersona  
FROM Persone as P  
WHERE P.Eta < 30
```

NPersona	RPersona
Andrea	21
Aldo	15
Filipo	30

Clausola FROM

Trovare il reddito e il telefono del responsabile in amministrazione.

Reparti

Reparto	Responsabile	Telefono
Logistica	Andrea	0459250890
Vendite	Franco	0469233892
Amministrazione	Maria	0454250708

Persone

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filipo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

```
SELECT Reddito, Telefono  
FROM Reparti, Persona  
WHERE ... ;
```

Senza ambiguità sui nomi:
può essere scritta in modo più compatto

Clausola WHERE

WHERE condition

condition è un'espressione booleana combinando condizioni semplici C_i con gli operatori **AND**, **OR**, **NOT**.

1. $C_i := \text{expr } [\theta \{ \text{expr} \mid \text{const} \}]$

dove

- $\text{expr} :=$ espressione che contiene riferimenti agli attributi delle tabelle che compaiono nella clausola FROM.
- $\theta \in \{=, <, >, <=, >=\}$
- $\text{const} :=$ valore dei domini di base.

2. Una riga soddisfa **condition** se l'espressione è vera quando valutata con i valori della riga.
3. Una riga che non soddisfa condition è scartata dal risultato finale.



Clausola WHERE

```
SELECT Nome, Reddito  
FROM Persone  
WHERE Età < 30;
```

```
SELECT Nome, Reddito  
FROM Persone  
WHERE Età < 30 OR Reddito > 50;
```

Operatore LIKE

```
WHERE attributo [NOT] LIKE 'pattern';
```

Nella clausola **WHERE** può apparire l'operatore **LIKE** per il confronto di stringhe. **LIKE** è un operatore di pattern matching. I pattern si costruiscono con i caratteri speciali **_** e **%**:

- **_** = 1 carattere qualsiasi.
- **%** = 0 o più caratteri qualsiasi.

```
SELECT Nome, Reddito  
FROM Persona  
WHERE Nome LIKE 'A%a';
```

Nome	Reddito
Andrea	21
Anna	35

Operatore SIMILAR TO

L'operatore **SIMILAR TO** è un **LIKE** più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).

► Esempi di componenti di espressioni regolari:

- **_** = 1 carattere qualsiasi.
- **%** = 0 o più caratteri qualsiasi.
- ***** = ripetizione del precedente match 0 o più volte.
- **+** = ripetizione del precedente match UNA o più volte.
- **{n,m}** = ripetizione del precedente match almeno n e non più di m volte.
- **[...]** = ... è un elenco di caratteri ammissibili

```
SELECT *  
FROM Persone  
WHERE Nome SIMILAR TO '[AFO]';
```

Per trovare nomi che
iniziano con A, F o O

Operatore BETWEEN

`WHERE` attributo `[NOT] BETWEEN` valore_iniziale `AND` valore_finale;

Nella clausola `WHERE` può apparire l'operatore `BETWEEN` per testare l'appartenenza di un valore ad un intervallo (non limitato ai valori numerici, usato anche con date e stringhe).

```
SELECT *  
FROM Persone  
WHERE Età BETWEEN 20 AND 40;
```

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Filipo	26	30
Olga	30	41



Operatore IN

```
WHERE attributo [NOT] IN ( valore [, ...] );
```

Nella clausola **WHERE** può apparire l'operatore **[NOT] IN** per testare l'appartenenza di un valore ad un insieme.

```
SELECT *  
FROM Persone  
WHERE Nome NOT IN ( 'Andrea', 'Franco' );
```

Operatore IS NULL

```
WHERE attributo IS [ NOT ] NULL;
```

Nella clausola **WHERE** può apparire l'operatore **IS [NOT] NULL** per testare se un valore è **NOT KNOWN (NULL)** o no.

Il valore NULL non rappresenta lo zero o una stringa vuota, ma l'assenza di dati o un valore sconosciuto.

A partire dalle versioni SQL-2 è stata adottata una **logica a tre valori** (true, false, unknown) e i valori nulli non sono considerati né veri, né falsi.

NON SI PUÒ usare '=' o '<>' con il valore NULL!

```
SELECT *  
FROM Persone  
WHERE Reddito IS NOT NULL;
```



Operatore ORDER BY

```
ORDER BY attributo [{ ASC | DESC }] [, ... ];
```

La clausola **ORDER BY** ordina le tuple del risultato in ordine rispetto agli attributi specificati,

ASCendente è lo standard.

```
SELECT Nome  
FROM Persone  
ORDER BY Nome DESC;
```

Operatori di aggregazione

Sono operatori che permettono di determinare un valore considerando i valori ottenuti da una SELECT.

Due tipi principali:

- COUNT.
- MAX, MIN, AVG, SUM.

```
SELECT MAX(Reddito)  
FROM Persone;
```

Quando si usano gli operatori aggregati, dopo la **SELECT** non possono comparire espressioni che usano i valori presenti nelle singole tuple perché il risultato è sempre e solo una tupla.

```
SELECT MAX(Reddito), Nome  
FROM Persone;
```

NO!



COUNT

```
COUNT({ * | expr | ALL expr | DISTINCT expr }])
```

Restituisce il numero di tuple significative nel risultato dell'interrogazione.

Tre casi comuni:

- **COUNT(*)** ritorna il numero di tuple nel risultato dell'interrogazione.
- **COUNT(expr)** ritorna il numero di tuple in ciascuna delle quali il valore expr è non nullo.
- **COUNT(ALL expr)** include duplicati e valori nulli.
- **COUNT(DISTINCT expr)** come con **COUNT(expr)** ma con l'ulteriore condizione che i valori di expr siano distinti.



COUNT

```
SELECT COUNT (ALL Età) FROM Persone;
```

```
SELECT COUNT (Età) FROM Persone;
```

Danno lo stesso risultato

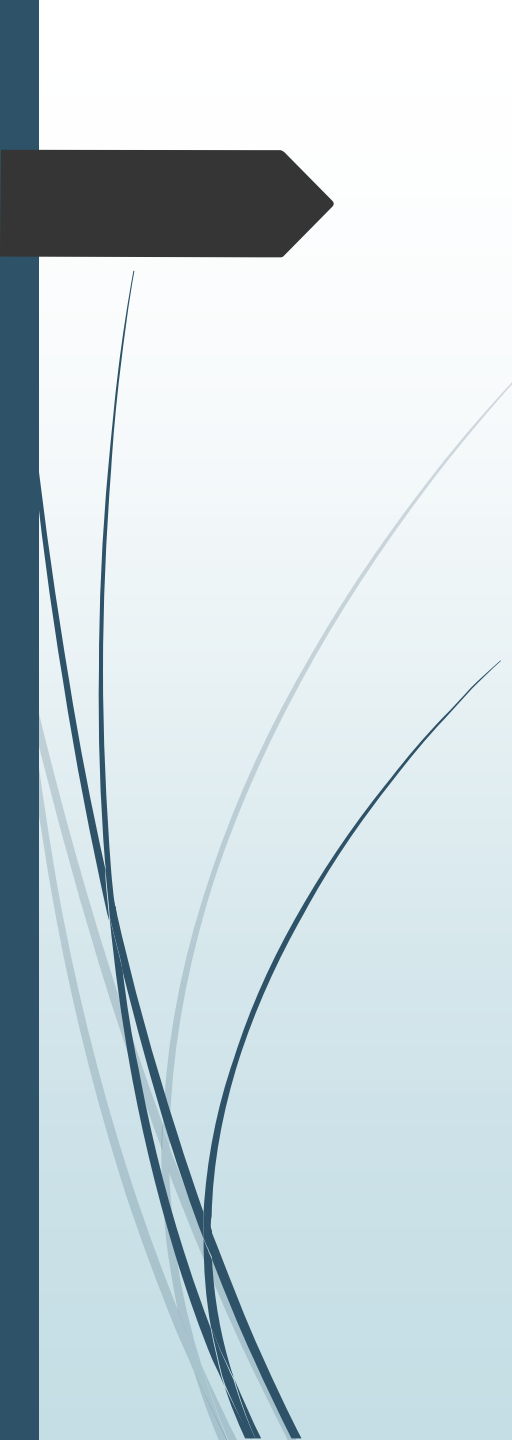
SUM/MAX/MIN/AVG

op ({ expr | DISTINCT expr })

Determinano un valore numerico (SUM/AVG) o alfanumerico (MAX/MIN) considerando le tuple significative nel risultato dell'interrogazione.

- op = SUM | AVG | MAX | MIN;
- expr è un'espressione che usa uno o più attributi e funzioni di attributi.
- DISTINCT considero solo i valori distinti.

```
SELECT AVG(Età) AS Media_Età  
FROM Persone;
```

Interrogazioni con raggruppamento: GROUP BY

- Un raggruppamento è un insieme di tuple che hanno medesimi valori su uno o più attributi caratteristici del raggruppamento.
- La clausola **GROUP BY attr [, ...]** permette di determinare tutti i raggruppamenti delle tuple della relazione risultato (tuple selezionate con la clausola **WHERE**) in funzione degli attributi dati.
- In una interrogazione che usa **GROUP BY**, la clausola **SELECT** contiene solamente gli attributi (da 0 a tutti) utilizzati per il raggruppamento ed eventuali funzioni di aggregazione sugli altri attributi.

Interrogazioni con raggruppamento

Visualizzare tutte le città raggruppate della tabella Studente

```
SELECT città  
FROM studente  
GROUP BY città
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	Via nobile	Verona	27.50
IN0002	Paoli	Paolo	Via dritta	Padova	28.6666
IN0003	Bianchi	Dante	Via storta	VERONA	18.345
IN0004	Rossi	Matteo	Via nobile	Verona	24.567
IN0005	Verdi	Luca	Via nuova	Va	28.6666
IN0006	Nocasa	Caio			

città
Padova
Verona
VERONA
Va

Interrogazioni con raggruppamento

Visualizzare tutte le città e i cognomi raggruppati.

```
SELECT cognome , LOWER(città)
FROM Studente
GROUP BY cognome , LOWER(città);
```

cognome	città
Verdi	Va
Rossi	verona
Paoli	padova
Nocasa	
Bianchi	verona

NOTA: nel GROUP BY non si possono usare espressioni con operatori di aggregazione.

LOWER: converte le stringhe in lowercase



Interrogazioni con raggruppamento

Nota: nella SELECT con GROUP BY non si possono specificare attributi che non sono raggruppati.

```
SELECT cognome, città  
FROM Studente  
GROUP BY città;
```

```
ERROR: COLUMN  
"studente.cognome" must  
appear IN the GROUP BY clause  
OR be used IN an aggregate  
function
```



Interrogazioni con raggruppamento

- La clausola **WHERE** permette di selezionare le righe che devono far parte del risultato.
- La clausola **HAVING** permette di selezionare i raggruppamenti che devono far parte del risultato.
- La sintassi è **HAVING bool_expr**, dove **bool_expr** è un'espressione booleana che può usare gli attributi usati nel **GROUP BY** e/o gli altri attributi mediante operatori di aggregazione.

Interrogazioni con raggruppamento

Visualizzare tutte le città raggruppate che iniziano con 'V' della tabella Studente.

```
SELECT città  
FROM Studente  
GROUP BY città  
HAVING città LIKE 'V%';
```

Città
Verona
VERONA
Va

Interrogazioni con raggruppamento

Visualizzare tutte le città e i cognomi raggruppati con almeno due studenti con lo stesso cognome.

```
SELECT cognome , LOWER(città)
FROM Studente
GROUP BY cognome , LOWER (città)
HAVING COUNT ( cognome ) >1;
```

cognome	città
Rossi	verona

Interrogazioni con raggruppamento

Si possono specificare espressioni con operatori di aggregazione su attributi non raggruppati anche nella clausola **HAVING**.

```
SELECT LOWER(città) AS "Città", AVG (media)::DECIMAL(5,2) AS  
"mediaArr."  
FROM Studente  
GROUP BY LOWER(città)  
HAVING AVG(media) >23.47 ;
```

Città	mediaArr
padova	28.67
va	28.67
Verona	23.47

Perchè il risultato contiene anche la tupla (verona, 23.47) quando la condizione HAVING non ammette tale valore?

$\text{AVG}(\text{media}) = 23.4712 \rightarrow 23.4712::\text{DECIMAL}(5,2) \rightarrow 23.47$



Interrogazioni con JOIN

```
TABLE1 join_type TABLE2 ON join_condition [, ...].
```

INNER JOIN, LEFT [OUTER] JOIN, RIGHT [OUTER] JOIN e FULL [OUTER] JOIN.

dove `join_condition` è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato.

Le tuple selezionate possono essere poi filtrate con la condizione della clausola **WHERE**.

Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**.

Interrogazioni con JOIN

```
table_name [ NATURAL ] join_type table_name [ON join_condition [, ...]].
```

dove `join_condition` è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola `WHERE`.

Interrogazioni con INNER JOIN

```
table1 [ INNER ] JOIN table2 ON join_condition [, ...]
```

Rappresenta il tradizionale θ join dell'algebra relazionale.

Combina ciascuna riga $r1$ di `table1` con ciascuna riga di `table2` che soddisfa la condizione della clausola **ON**.

Interrogazioni con LEFT OUTER JOIN

```
table1 LEFT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un **INNER JOIN**. Poi, per ciascuna riga $r1$ di `table1` che non soddisfa la condizione con qualsiasi riga di `table2`, si aggiunge una riga al risultato con i valori di $r1$ e assegnando **NULL** agli altri attributi..

Il **LEFT [OUTER] JOIN** non è simmetrico!

Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!



Interrogazioni con RIGHT OUTER JOIN

```
table1 RIGHT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un **INNER JOIN**. Poi, per ciascuna riga r2 di table2 che non soddisfa la condizione con qualsiasi riga di table1, si aggiunge una riga al risultato con i valori di r2 e assegnando **NULL** agli altri attributi..

Interrogazioni con FULL OUTER JOIN

```
table1 FULL [ OUTER ] JOIN table2 ON join_condition [, ...].
```

- È equivalente a:
INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.
- Non è equivalente a CROSS JOIN!